Modular API
User Guide

MAPI-01

July 2026

Version 2.0

Table of Contents

- Preface

1. General Information
    - Key Features
    - Architecture Components
    - How It Works
    - Available Modules
2. Installation and Configuration
    - Prerequisites
    - Installation Steps
    - Configuration
    - Configuration Parameters
    - SaaS Mode (AWS DynamoDB)
    - OnPrem Mode (MongoDB)
3. Policies Management
    - Policy Commands
    - Policy Structure
    - Policy Rules
    - Resource Syntax
    - Real-World Policy Examples
    - Creating Policies
    - Viewing Policies
    - Updating Policies
    - Deleting Policies
4. Group Management
    - Group Commands
    - Real-World Group Examples
    - Managing Group Policies
    - Viewing Groups
    - Deleting Groups
    - Complete Group Setup Example
5. User Management
    - User Commands
    - Creating Users
    - Real-World User Setup
    - Managing User Groups
    - Password Management
    - Blocking and Unblocking Users
    - User Meta Attributes
    - Viewing Users
    - Deleting Users
6. Modules Installation
    - Module Requirements
    - Module Commands
    - Installing M3Admin Modules
    - Uninstalling Modules
    - Dependency Management
7. Audit Service
    - Audit Commands

    - Audit Output Example
    - Real-World Audit Examples
8. First Run
    - Pre-Flight Checklist
    - Starting the Server
    - Verify Server Health
    - Access Swagger Documentation
    - Using API Clients
9. Modular API Schema
    - Architecture Diagram
    - Request Flow
    - Database Schema
10. Annexes
    - Annex 1: Common Use Cases
    - Annex 2: Developers Guide
11. Project Information

Preface

About This Guide

This guide describes the installation, configuration, and administration of Modular API – a unified facade server that provides centralized management for multiple modules through a single entry point. It enables unified authentication, authorization, and audit capabilities across all integrated services.

Target Audience

This guide is designed for:

- **System Administrators** responsible for deploying and managing Modular API infrastructure
- **Security Engineers** configuring policies, groups, and user access controls
- **DevOps Engineers** integrating Modular API into CI/CD pipelines
- **Developers** extending Modular API with custom modules

Related Documents

- For M3 Admin CLI usage and commands, refer to M3 Admin User Guide
- For Modular CLI configuration and usage, see Modular-CLI README
- For Modular SDK integration, see Modular-SDK Documentation

1. General Information

Modular API is a unified facade server that allows combining different services controls under one custom API/CLI service. It provides centralized management for multiple modules through a single entry point with unified authentication, authorization, and audit capabilities.

Key Features

- **Unified Authentication**: Single JWT-based authentication mechanism across all modules
- **ABAC Authorization**: Attribute-Based Access Control with policies, groups, and users
- **Pre-validation**: Request validation before forwarding to module backends
- **Centralized Audit**: Complete logging and audit trail of all operations
- **Modular Architecture**: Easy integration of new services/modules
- **CLI Interface**: Command-line tool (`modular-cli`) for all operations

Architecture Components

Modular API provides a unified facade for multiple backend services:

- **API Layer**: API Gateway, Lambda Function URL, Container exposed endpoints. Message queues (AMQP/RabbitMQ) for asynchronous operations. Direct HTTP/HTTPS connections to backend servers
- **Authentication**: Custom JWT / Cognito
- **Authorization**: Policy-based ABAC (Attribute-Based Access Control) system
- **CLI**: Modular-CLI for command-line interactions
- **Backend Services**: Application servers (Java, Python, Node.js). Message brokers (RabbitMQ, AWS SQS). Cloud provider APIs (AWS, Azure, Google Cloud). Internal service APIs
- **Compute Options**: AWS Lambda, AWS Batch, EKS/Kubernetes, EC2 instances
- **Runtimes**: Python 3.14, Java 17, NodeJS 18.16.0
- **Persistence**: AWS DynamoDB, MongoDB Atlas, AWS DocumentDB, AWS RDS, S3

How It Works

The Modular API acts as a facade layer that:

1. Receives requests from CLI clients or API consumers
2. Authenticates users via JWT tokens
3. Validates permissions using ABAC policies
4. Pre-validates request parameters
5. Routes requests to appropriate module backends
6. Manages temporary tokens for module authentication
7. Records all operations in centralized audit log

Available Modules

Modular API supports multiple independent modules. In a typical M3Admin deployment, these modules are used:

Module	Description
m3admin	Core administrative functions for AWS, Azure, GCP, OpenStack, etc.
billing	Cost management, reports, budgets, pricing policies
chef	Configuration management with Chef integration
lowlevel	System-level operations and utilities
maintenance	System maintenance tasks
notifications	Email and notification management

Module	Description
permissions	User and access control management

Each module is installed separately and has its own policies and permissions.

Content ↑

2. Installation and Configuration

Prerequisites

Before installation, ensure you have:

- **Python 3.14**
- **pip** package manager
- **venv** or **virtualenv**

Download links:

- Python for Windows
- Python for Linux
- Python for Mac

IMPORTANT: Using a virtual environment is highly recommended to prevent dependency conflicts.

Installation Steps

1. Create Virtual Environment

On Linux/Mac:

```
python3.10 -m venv modular_api_venv
source modular_api_venv/bin/activate
```

On Windows:

```
python -m venv modular_api_venv
.\modular_api_venv\Scripts\activate
```

2. Install Modular API

```
# Install from source
pip install -e /path/to/modular-api/

# Verify installation
modular --version
```

3. Verify Installation

```
modular describe
```

Expected output:

```
Cannot access . Writing logs to C:\Users\some_user\.modular_api\log
Modular-API: 4.3.8
Modular-SDK: 7.1.4
Modular-CLI-SDK: 3.1.0
Installed modules:
chef                    1.0.2
m3admin                 4.170.0
stm                     5.9.0
```

Configuration

Environment File (.env)

Create `.env` file in `modular_api/` directory:

```
# Basic configuration
MODULAR_API_SECRET_KEY=your-secure-passphrase-here
MODULAR_API_MODE=saas
MODULAR_API_CALLS_PER_SECOND_LIMIT=30
MODULAR_API_MIN_CLI_VERSION=2.0
MODULAR_API_ENABLE_PRIVATE_MODE=false

# Logs configuration
MODULAR_API_SERVER_LOG_LEVEL=INFO
MODULAR_API_CLI_LOG_LEVEL=INFO
MODULAR_API_LOG_PATH=/var/log/modular

# For onprem/private mode - MongoDB configuration
MODULAR_API_MONGO_URI=mongodb://localhost:27017
MODULAR_API_MONGO_DATABASE=modular-api
MODULAR_API_RATE_LIMITS_MONGO_DATABASE=modular-api-rate-limits

# For onprem/private mode - Vault configuration
MODULAR_CLI_VAULT_TOKEN=token
MODULAR_CLI_VAULT_ADDR=http://127.0.0.1:8200
```

Configuration Parameters

Basic Configuration

Parameter	Description	Example
MODULAR_API_SECRET_KEY	Passphrase for JWT token encryption and hash calculation	your-secure-passphrase
MODULAR_API_MODE	Deployment mode (saas or onprem)	saas
MODULAR_API_CALLS_PER_SECOND_LIMIT	Rate limiting (requests per second)	30
MODULAR_API_MIN_CLI_VERSION	Minimum supported CLI version	2.0
MODULAR_API_ENABLE_PRIVATE_MODE	Enable private mode (uses MongoDB)	false

Logging Configuration

Parameter	Description	Default
MODULAR_API_SERVER_LOG_LEVEL	Server log verbosity	INFO
MODULAR_API_CLI_LOG_LEVEL	CLI log verbosity	INFO
MODULAR_API_LOG_PATH	Log file storage path	%USERPROFILE%\.modular_api\log

Database Configuration (OnPrem Mode)

Parameter	Description	Example
MODULAR_API_MONGO_URI	MongoDB connection string	mongodb://localhost:27017
MODULAR_API_MONGO_DATABASE	Database name for collections	modular-api
MODULAR_API_RATE_LIMITS_MONGO_DATABASE	Database name for rate limiting	modular-api-rate-limits

Vault Configuration (OnPrem/Private Mode)

Parameter	Description	Example
MODULAR_CLI_VAULT_TOKEN	Vault authentication token	token

Parameter	Description	Example
MODULAR_CLI_VAULT_ADDR	Vault server address	http://127.0.0.1:8200

SaaS Mode (AWS DynamoDB)

Set environment variables:

```
export AWS_ACCESS_KEY_ID=<your_access_key>
export AWS_SECRET_ACCESS_KEY=<your_secret_key>
export AWS_SESSION_TOKEN=<your_session_token>
export AWS_REGION=<your_region>
```

OnPrem Mode (MongoDB)

Ensure MongoDB is running and accessible, then configure the connection URI in `.env` file.

NOTE: You can export environment variables instead of using the `.env` file.

Content ↑

3. Policies Management

Policies define permissions for groups and users. They specify which commands and resources are allowed or denied.

Policy Commands

```
# Add new policy
modular policy add --policy <POLICY_NAME> --policy_path <PATH_TO_POLICY.json>

# Update existing policy
modular policy update --policy <POLICY_NAME> --policy_path <PATH_TO_POLICY.json>

# Describe policies
modular policy describe                      # List all policies
modular policy describe --policy <NAME>      # Describe specific policy
modular policy describe --expand             # Show all policies with content

# Delete policy
modular policy delete --policy <POLICY_NAME>
```

Policy Structure

```
[
  {
    "Effect": "Allow",
    "Module": "m3admin",
    "Resources": [
      "aws",
      "azure",
      "billing",
      "tenant",
      "region"
    ]
  },
  {
    "Effect": "Deny",
    "Module": "billing",
    "Resources": [
      "close_month"
    ]
  }
]
```

Policy Rules

Property	Description	Required
Effect	Must be Allow or Deny. Deny takes precedence	✓
Module	Module name (e.g., m3admin, billing, chef) or *	✓
Resources	List of commands, groups, or wildcards	✓

Key Rules:

- **Deny** effect has more priority than **Allow**
- If commands/groups/subgroups/modules are not in user policy(ies), they will not be available
- For m3admin root(src) module use `m3admin` name
- Use * in “Module” property to apply “Effect” to all installed modules

- Module name equals the `module_name` property in `api_module.json` file

Resource Syntax

Syntax	Description
*	All commands in the module
command_name	Specific command
group:*	All commands in a group
group:command	Specific command in a group
group/subgroup:*	All commands in a subgroup
group/subgroup:command	Specific command in a subgroup

Real-World Policy Examples

L3 Support Policy

This policy grants L3 support engineers access to diagnostic and management commands:

```
[
  {
    "Effect": "Allow",
    "Module": "m3admin",
    "Resources": [
      "aws",
      "azure",
      "billing",
      "configure",
      "enterprise",
      "environment",
      "google",
      "hardware",
      "maintenance",
      "notifications",
      "openstack",
      "paas",
      "permissions",
      "policy",
      "region",
      "security",
      "tenant",
      "timeline",
      "vclouddirector",
      "workspace",
      "yandex",
      "get_operation_status"
    ]
  },
  {
    "Effect": "Allow",
    "Module": "paas",
    "Resources": [
      "activate",
      "chef",
      "deactivate",
      "delete_init_script",
      "delete_service_parameter",
```

```
        "describe",
        "describe_init_script",
        "describe_service_parameter",
        "register_service_parameter",
        "upload_init_script"
      ]
    },
    {
      "Effect": "Allow",
      "Module": "billing",
      "Resources": [
        "budget",
        "export_aws",
        "export_azure",
        "pricing_policy",
        "health_check"
      ]
    }
  ]
```

Administrator Policy

Full access to all modules:

```
  [
    {
      "Effect": "Allow",
      "Module": "m3admin",
      "Resources": ["*"]
    },
    {
      "Effect": "Allow",
      "Module": "billing",
      "Resources": ["*"]
    },
    {
      "Effect": "Allow",
      "Module": "chef",
      "Resources": ["*"]
    },
    {
      "Effect": "Allow",
      "Module": "notifications",
      "Resources": ["*"]
    },
    {
      "Effect": "Allow",
      "Module": "permissions",
      "Resources": ["*"]
    }
  ]
```

Read-Only Policy

View-only access:

```
  [
    {
      "Effect": "Allow",
```

```
      "Module": "m3admin",
      "Resources": [
        "tenant:describe",
        "region:describe",
        "aws:describe_security_groups",
        "azure:list_storage"
      ]
    },
    {
      "Effect": "Allow",
      "Module": "billing",
      "Resources": [
        "describe_business_units",
        "describe_tenant_business_unit"
      ]
    }
  ]
```

CICD Policy

For automated deployment pipelines:

```
  [
    {
      "Effect": "Allow",
      "Module": "m3admin",
      "Resources": [
        "configure:create_audit_and_rpc_queues",
        "configure:activate_tenant_guardduty",
        "aws:activate",
        "azure:activate",
        "tenant:describe"
      ]
    },
    {
      "Effect": "Allow",
      "Module": "permissions",
      "Resources": [
        "add_group",
        "add_user",
        "assign_position"
      ]
    }
  ]
```

Creating Policies

Example 1: Create L3 Support Policy

```
modular policy add \
  --policy "m3admin-l3support" \
  --policy_path "/opt/policies/l3support_policy.json"
```

Example 2: Create Multiple Module Policies

```
# Root module policies
modular policy add --policy "m3admin-administrator" \
  --policy_path "/opt/policies/m3admin_root/administrator_policy.json"

modular policy add --policy "m3admin-readonly" \
```

```
    --policy_path "/opt/policies/m3admin_root/readonly_policy.json"

  modular policy add --policy "m3admin-cicd" \
    --policy_path "/opt/policies/m3admin_root/cicd_policy.json"

  # Billing module policies
  modular policy add --policy "billing-administrator" \
    --policy_path "/opt/policies/m3admin_billing/administrator_policy.json"

  modular policy add --policy "billing-readonly" \
    --policy_path "/opt/policies/m3admin_billing/readonly_policy.json"
```

Viewing Policies

```
# List all policies
modular policy describe

# View specific policy with full content
modular policy describe --policy "m3admin-l3support"

# Show all policies with content
modular policy describe --expand

# Export policy to JSON
modular policy describe --policy "m3admin-l3support" --json > policy_backup.json
```

Note: The `--expand` flag is automatically enabled when `--policy` is specified, so `modular policy describe --policy "name" --expand` is equivalent to `modular policy describe --policy "name"`. The flag is only useful when describing all policies without specifying a policy name.

Updating Policies

```
modular policy update \
  --policy "m3admin-l3support" \
  --policy_path "/opt/policies/l3support_policy_v2.json"
```

Deleting Policies

```
modular policy delete --policy "m3admin-deprecated"
```

WARNING: Deleting a policy affects all groups using it. Ensure no active groups reference the policy before deletion.

Content ↑

4. Group Management

Groups combine multiple policies to define role-based permissions. Users are assigned to groups to inherit their permissions.

Group Commands

```
# Create group with policies
modular group add --group <GROUP_NAME> --policy <POLICY_1> --policy <POLICY_2>

# Add policy to existing group
modular group add_policy --group <GROUP_NAME> --policy <POLICY_NAME>

# Remove policy from group
modular group delete_policy --group <GROUP_NAME> --policy <POLICY_NAME>

# Describe groups
modular group describe                    # List all groups
modular group describe --group <NAME>     # Describe specific group

# Delete group
modular group delete --group <GROUP_NAME>
```

Real-World Group Examples

Create Administrator Group

Full system access with all module permissions:

```
modular group add --group "administrators" \
  --policy "m3admin-administrator" \
  --policy "billing-administrator" \
  --policy "chef-administrator" \
  --policy "notifications-administrator" \
  --policy "permissions-administrator"
```

Create L3 Support Group

Diagnostic and management access:

```
modular group add --group "l3support" \
  --policy "m3admin-l3support" \
  --policy "billing-l3support" \
  --policy "chef-l3support" \
  --policy "notifications-l3support" \
  --policy "permissions-l3support"
```

Create Read-Only Group

View-only access for auditors:

```
modular group add --group "readonly" \
  --policy "m3admin-readonly" \
  --policy "billing-readonly" \
  --policy "chef-readonly" \
  --policy "notifications-readonly" \
  --policy "permissions-readonly"
```

Create CICD Group

Automated deployment access:

```
modular group add --group "cicd" \
  --policy "m3admin-cicd" \
  --policy "billing-cicd" \
  --policy "notifications-cicd" \
  --policy "permissions-cicd"
```

Create Billing Team Group

Financial operations access:

```
modular group add --group "billing" \
  --policy "billing-billing" \
  --policy "m3admin-readonly"
```

Create M3 Server Group

Service-to-service communication:

```
modular group add --group "m3server" \
  --policy "m3admin-m3server" \
  --policy "billing-m3server" \
  --policy "notifications-m3server" \
  --policy "permissions-m3server"
```

Managing Group Policies

Add Additional Policies

```
# Add maintenance policy to l3support group
modular group add_policy --group "l3support" --policy "maintenance-l3support"

# Add low-level policy to administrators
modular group add_policy --group "administrators" --policy "lowlevel-administrator"
```

Remove Policies

```
modular group delete_policy --group "readonly" --policy "billing-readonly"
```

Viewing Groups

```
# List all groups
modular group describe

# View specific group
modular group describe --group "l3support"

# Export group configuration
modular group describe --group "administrators" --json > admin_group.json
```

Expected output:

```
$ modular group describe --json
[
    {
        "Group name": "admin_group",
        "State": "activated",
        "Policy(ies)": [
            "admin_policy"
        ],
        "Last modification date": null,
        "Creation date": "17.04.2024 18:14:03",
        "Consistency status": "OK"
    },
```

```
    {
        "Group name": "cicd_group",
        "State": "activated",
        "Policy(ies)": [
            "admin_policy"
        ],
        "Last modification date": null,
        "Creation date": "12.06.2024 08:18:45",
        "Consistency status": "OK"
    },
    {
        "Group name": "chef_group",
        "State": "activated",
        "Policy(ies)": [
            "policy-chef"
        ],
        "Last modification date": null,
        "Creation date": "11.11.2025 15:56:24",
        "Consistency status": "OK"
    }
]
```


Deleting Groups

```
modular group delete --group "deprecated-group"
```

WARNING: Deleting a group immediately affects all users assigned to it. They will lose associated permissions.

Complete Group Setup Example

Typical production environment setup:

```
#!/bin/bash
# Administrator group - full access
modular group add --group "administrators" \
  --policy "lowlevel-administrator" \
  --policy "m3admin-administrator" \
  --policy "billing-administrator" \
  --policy "maintenance-administrator" \
  --policy "notifications-administrator" \
  --policy "permissions-administrator" \
  --policy "chef-administrator"

# L3 Support group - operational access
modular group add --group "l3support" \
  --policy "lowlevel-l3support" \
  --policy "m3admin-l3support" \
  --policy "billing-l3support" \
  --policy "maintenance-l3support" \
  --policy "notifications-l3support" \
  --policy "permissions-l3support" \
  --policy "chef-l3support"

# Billing team - financial operations
modular group add --group "billing" \
  --policy "lowlevel-billing" \
  --policy "billing-billing" \
  --policy "maintenance-billing"
```

```
# Read-only group - auditors and viewers
modular group add --group "readonly" \
  --policy "m3admin-readonly" \
  --policy "billing-readonly" \
  --policy "maintenance-readonly" \
  --policy "permissions-readonly" \
  --policy "chef-readonly" \
  --policy "notifications-readonly"

# CICD group - automated deployments
modular group add --group "cicd" \
  --policy "lowlevel-cicd" \
  --policy "m3admin-cicd" \
  --policy "billing-cicd" \
  --policy "maintenance-cicd" \
  --policy "notifications-cicd" \
  --policy "permissions-cicd" \
  --policy "chef-cicd"

# M3 Server group - inter-service communication
modular group add --group "m3server" \
  --policy "lowlevel-m3server" \
  --policy "m3admin-m3server" \
  --policy "billing-m3server" \
  --policy "notifications-m3server" \
  --policy "permissions-m3server"

echo "All groups created successfully"
```

Content ↑

5. User Management

Users authenticate to Modular API and inherit permissions from assigned groups.

User Commands

```
# Create user
modular user add --username <NAME> --group <GROUP> [--password <PWD>]

# Manage groups
modular user add_to_group --username <NAME> --group <GROUP1> --group <GROUP2>
modular user remove_from_group --username <NAME> --group <GROUP>

# Manage credentials
modular user change_password --username <NAME> --password <NEW_PWD>
modular user change_username --old_username <OLD> --new_username <NEW>

# Block/Unblock users
modular user block --username <NAME> --reason <REASON>
modular user unblock --username <NAME> --reason <REASON>

# Manage meta attributes
modular user set_meta_attribute --username <NAME> --key <KEY> --value <VAL>
modular user update_meta_attribute --username <NAME> --key <KEY> --value <VAL>
modular user delete_meta_attribute --username <NAME> --key <KEY>
modular user reset_meta --username <NAME>
modular user get_meta --username <NAME>

# Describe users
modular user describe                       # List all users
modular user describe --username <NAME>     # Describe specific user

# Delete user
modular user delete --username <NAME>
```

Creating Users

Example 1: Create Administrator

```
modular user add \
  --username "admin_user" \
  --group "administrators" \
  --password "SecureAdminPass123!"
```

Example 2: Create User with Auto-Generated Password

```
modular user add --username "support_engineer" --group "l3support"
```

Output:

```
Autogenerated password: KcuE6V3tPLqEWvbr
PAY ATTENTION: You can get the user password only when you add the new user. You cannot
retrieve it later.
If you lose it, you must create a new user or change password via 'modular user
change_password' command
User 'support_engineer' has been successfully activated.
User added to the following group(s):
l3support
```

CRITICAL: Auto-generated passwords cannot be retrieved after creation. Save them immediately.

Example 3: Create User with Multiple Groups

```
modular user add \
  --username "ops_manager" \
  --group "l3support" \
  --group "readonly"
```

Real-World User Setup

Complete user creation script:

```
#!/bin/bash

# Create administrator
modular user add --username "admin" --group "administrators"

# Create L3 support engineers
modular user add --username "support_john" --group "l3support"
modular user add --username "support_jane" --group "l3support"

# Create billing analysts
modular user add --username "billing_analyst" --group "billing"

# Create read-only auditors
modular user add --username "auditor_external" --group "readonly"

# Create CICD service account
modular user add --username "cicd-bot" --group "cicd" --password "CI/CD-Secure-Token-2025"

# Create M3 server service account
modular user add --username "m3server-svc" --group "m3server"

echo "All users created successfully"
```

Managing User Groups

Add Groups to User

```
# Add multiple groups
modular user add_to_group \
  --username "ops_manager" \
  --group "l3support" \
  --group "billing" \
  --group "readonly"
```

Remove Groups from User

```
modular user remove_from_group \
  --username "ops_manager" \
  --group "billing"
```

Password Management

Change Password

```
modular user change_password \
  --username "support_john" \
  --password "NewSecurePassword456!"
```

Change Username

```
modular user change_username \
  --old_username "john_doe" \
  --new_username "john_doe_contractor"
```

Blocking and Unblocking Users

Block User

```
modular user block \
  --username "suspicious_user" \
  --reason "Security investigation in progress - Ticket #INC-12345"
```

Unblock User

```
modular user unblock \
  --username "suspicious_user" \
  --reason "Security investigation completed - cleared by SecOps"
```

User Meta Attributes

Meta attributes restrict parameter values for users, providing fine-grained access control.

Restrict Regional Access

```
# Allow user to work only in specific regions
modular user set_meta_attribute \
  --username "support_regional" \
  --key "region" \
  --value "eu-central-1" \
  --value "eu-west-1"   # Multiple --value flags
```

Now `support_regional` can only use `--region eu-central-1` or `--region eu-west-1` in commands.

Restrict Tenant Access

```
# Allow user to manage only specific tenants
modular user set_meta_attribute \
  --username "ops_limited" \
  --key "tenant_name" \
  --value "production-aws-eu" \
  --value "staging-aws-eu"
```

Restrict Cloud Provider

```
modular user set_meta_attribute \
  --username "aws_specialist" \
  --key "cloud" \
  --value "AWS"
```

Store Custom User Data

```
# Store auxiliary data (department, cost center, etc.)
modular user set_meta_attribute \
  --username "billing_analyst" \
  --meta_type "aux_data" \
  --key "department" \
  --value "Finance"

modular user set_meta_attribute \
  --username "billing_analyst" \
  --meta_type "aux_data" \
  --key "cost_center" \
```

```
  --value "CC-12345"

# Store service name mappings
modular user set_meta_attribute \
  --username "service_user" \
  --meta_type "aux_data" \
  --key "service_name" \
  --value_as_json '{"MOBILE": "MOB", "DIAL": "AI"}'  # JSON format
```

Update Meta Attributes

```
# Update existing meta attribute
modular user update_meta_attribute \
  --username "support_regional" \
  --key "region" \
  --value "eu-central-1" \
  --value "eu-west-1" \
  --value "us-east-1"
```

View User Meta Attributes

```
modular user get_meta --username "support_regional"
```

Output:

```
{
  "allowed_values": {
    "region": ["eu-central-1", "eu-west-1", "us-east-1"],
    "tenant_name": ["production-aws-eu"]
  },
  "aux_data": {
    "department": "Operations",
    "team": "Platform",
    "service_name": {
      "MOBILE": "MOB",
      "DIAL": "AI"
    }
  }
}
```

Delete Specific Meta Attributes

```
modular user delete_meta_attribute \
  --username "support_regional" \
  --key "region"
```

Reset All Meta Attributes

```
modular user reset_meta --username "support_regional"
```

Viewing Users

```
# List all users
modular user describe

# View specific user
modular user describe --username "support_john"

# Export user list
modular user describe --json > users_backup.json
```

Expected output:

```
$ modular user describe --json
User(s) description
[
    {
        "Username": "admin",
        "Groups": [
            "admin_group"
        ],
        "State": "activated",
        "State reason": null,
        "User meta": {},
        "Modification date": null,
        "Creation Date": "17.04.2024 18:15:24",
        "Consistency status": "OK"
    },
    {
        "Username": "support_engineer",
        "Groups": [
            "admin_group"
        ],
        "State": "activated",
        "State reason": null,
        "User meta": {},
        "Modification date": null,
        "Creation Date": "17.11.2025 15:35:12",
        "Consistency status": "OK"
    }
]

$ modular user describe --username admin --json
[
    {
        "Username": "admin",
        "Groups": [
            "admin_group"
        ],
        "State": "activated",
        "State reason": null,
        "User meta": {},
        "Modification date": null,
        "Creation Date": "17.04.2024 18:15:24",
        "Consistency status": "OK"
    }
]
```

Deleting Users

```
modular user delete --username "contractor_expired"
```

WARNING: User deletion is permanent and cannot be undone.

Content ↑

6. Modules Installation

Modules extend Modular API functionality. Each module is an independent package that can be installed separately. The most commonly used module is **m3admin** (core administrative functions), which is typically deployed alongside other modules like billing, chef, and notifications.

Module Requirements

Each module must have:

1. **api_module.json** file:

```
{
  "module_name": "m3admin",
  "cli_path": "/path/to/cli/main/group",
  "mount_point": "/",
  "dependencies": [
    {
      "module_name": "dependent_module",
      "min_version": "1.0.0"
    }
  ]
}
```

2. **setup.py** file in the same directory
3. **Proper naming convention**:
    - `groupname.py` for command groups
    - `groupname_subgroupname.py` for subgroups

Module Commands

```
# Install module
modular install --module_path /path/to/module/setup.py

# Uninstall module
modular uninstall --module_name <MODULE_NAME>

# Describe installed modules
modular describe
```

Installing M3Admin Modules

Complete Installation Script

From the CICD deployment:

```
#!/bin/bash

# Define paths
M3ADMIN_FOLDER="/usr/local/project/modular/m3admin"

# Install core module
echo "Installing m3admin core module..."
modular install --module_path ${M3ADMIN_FOLDER}/src

# Install billing module
echo "Installing billing module..."
modular install --module_path ${M3ADMIN_FOLDER}/billing
```

```
# Install chef configuration management
echo "Installing chef module..."
modular install --module_path ${M3ADMIN_FOLDER}/chef

# Install low-level operations module
echo "Installing lowlevel module..."
modular install --module_path ${M3ADMIN_FOLDER}/lowlevel

# Install maintenance module
echo "Installing maintenance module..."
modular install --module_path ${M3ADMIN_FOLDER}/maintenance

# Install notifications module
echo "Installing notifications module..."
modular install --module_path ${M3ADMIN_FOLDER}/notifications

# Install permissions module
echo "Installing permissions module..."
modular install --module_path ${M3ADMIN_FOLDER}/permissions

echo "All modules installed successfully"
```

Verify Installation

```
modular describe --table
```

Expected output:

```
Modular-API: 4.3.8
Modular-SDK: 7.1.4
Modular-CLI-SDK: 3.1.0
Installed modules
+-------------+---------+
| Module name | Version |
+-------------+---------+
|     chef    |  1.0.2  |
| ----------- | ------- |
|   m3admin   | 4.170.0 |
| ----------- | ------- |
|     stm     |  5.9.0  |
+-------------+---------+
```

Uninstalling Modules

```
# Uninstall specific module
modular uninstall --module_name "chef"

# Verify removal
modular describe
```

WARNING: Uninstalling a module removes all associated commands and may affect dependent modules.

Dependency Management

Modular API automatically checks dependencies during installation. If a required dependency is missing or has insufficient version, installation fails with a clear error message.

Content ↑

7. Audit Service

All successful command executions are automatically logged to the ModularAudit collection for compliance and tracking.

NOTE: “Describe” commands are not recorded in audit logs.

Audit Commands

```
# View recent audit (last 7 days)
modular audit

# Filter by date range
modular audit --from_date 2025-01-01 --to_date 2025-01-31

# Filter by group
modular audit --group "billing"

# Filter by command
modular audit --command "activate"

# Show more records
modular audit --limit 50

# Show only failed operations
modular audit --invalid

# Combine filters
modular audit \
  --from_date 2025-01-01 \
  --to_date 2025-01-31 \
  --group "billing" \
  --command "export_aws" \
  --limit 100

# Export to JSON
modular audit --from_date 2025-01-01 --to_date 2025-01-31 --json > audit_jan2025.json
```

Audit Output Example

```
$ modular audit --json

[
    {
        "Group": "policy",
        "Command": "add",
        "Timestamp": "11.11.2025 10:01:55",
        "Parameters": "{\"policy\": \"test-12345123\", \"policy_path\": \"policy2.json\"}",
        "Execution warnings": [],
        "Result": "Policy with name 'test-123' successfully added",
        "Consistency status": "OK"
    },
    {
        "Group": "user",
        "Command": "add",
        "Timestamp": "17.11.2025 15:35:12",
        "Parameters": "{\"username\": \"support_engineer\", \"group\": [\"admin_group\"],
\"password\": \"*****\"}",
        "Execution warnings": [],
```

```
        "Result": "User 'support_engineer' has been successfully activated.\r\nUser added to
the following group(s):\r\nadmin_group",
        "Consistency status": "OK"
    }
]
```

Real-World Audit Examples

Track User Activity

```
# View all actions by specific user
modular audit --from_date 2025-01-01 --limit 100 | grep "support_john"
```

Security Audit

```
# Track all permission changes
modular audit --group "permissions" --from_date 2025-01-01 --limit 200

# Track policy modifications
modular audit --group "policy" --from_date 2025-01-01 --limit 200

# Track user creations and modifications
modular audit --group "user" --from_date 2025-01-01 --limit 200
```

Compliance Reporting

```
# Monthly compliance report
modular audit \
  --from_date 2025-01-01 \
  --to_date 2025-01-31 \
  --limit 10000 \
  --json > compliance_report_jan2025.json
```

Failed Operations Analysis

```
# Show only failed operations
modular audit --invalid --limit 100

# Failed operations in specific period
modular audit \
  --invalid \
  --from_date 2025-01-20 \
  --to_date 2025-01-21 \
  --limit 50
```

Billing Operations Audit

```
# Track all billing operations
modular audit --group "billing" --limit 200

# Track cost-sensitive operations
modular audit --group "billing" --command "close_month" --limit 50
```

Content ↑

8. First Run

Pre-Flight Checklist

Before starting Modular API server:

1. ☑ `.env` file properly configured
2. ☑ Database mode set (`saas` or `onprem`)
3. ☑ Database credentials configured
4. ☑ At least one module installed
5. ☑ At least one policy created
6. ☑ At least one group created with attached policy
7. ☑ At least one user created and assigned to a group

Starting the Server

Using Gunicorn (Production)

```
modular run \
  --host 0.0.0.0 \
  --port 8086 \
  --prefix /integration \
  --gunicorn \
  --workers 2 \
  --worker_timeout 62 \
  --swagger \
  --swagger_prefix /swagger
```

Worker Count Guidelines:

Guideline	Description
Default formula	(2 × CPU_cores) + 1 (e.g., 8-core server = 17 workers)
Minimum	2 workers for high availability
Resource-constrained	2 workers as in production example above
Adjustment factor	Based on expected load and available memory
Trade-off	More workers = higher concurrency but more memory usage

Using Bottle (Development)

```
modular run \
  --host 127.0.0.1 \
  --port 8085 \
  --prefix /integration \
  --swagger \
  --swagger_prefix /swagger
```

NOTE: In Bash, the values `/integration` and `/swagger` are resolved as file paths. Therefore, you should use `//integration` and `//swagger` instead.

Expected output:

```
Bottle v0.12.25 server starting up (using WSGIRefServer())...
Listening on http://127.0.0.1:8085/
Hit Ctrl-C to quit.
```

Production Deployment as Systemd Service

Create service file `/etc/systemd/system/m3-modular-api.service`:

```
[Unit]
Description=Modular API service

[Service]
Restart=on-failure
RestartSec=5s
User=m3modular
Group=m3modular
WorkingDirectory=/usr/local/project/modular/m3-modular-admin/modular_api
EnvironmentFile=/usr/local/project/modular/conf/envVariablesForService
PrivateTmp=true
PermissionsStartOnly=true
ExecStart=/usr/local/project/modular/.api_venv/bin/modular run \
  --host 0.0.0.0 \
  --gunicorn \
  --port 8086 \
  --workers 2 \
  --worker_timeout 62 \
  --prefix /integration \
  --swagger \
  --swagger_prefix /swagger

[Install]
WantedBy=default.target
```

Environment file `/usr/local/project/modular/conf/envVariablesForService`:

```
PYTHONUNBUFFERED=1
VIRTUAL_ENV=/usr/local/project/modular/.api_venv
PATH=/usr/local/project/modular/.api_venv/bin
PYTHONPATH=/usr/local/project/modular/.api_venv/bin/

PROJECT_PATH=/usr/local/project/modular
POLICY_PATH=/usr/local/project/modular/conf/modular_policies/m3admin_policies

AWS_REGION=eu-central-1
AWS_DEFAULT_REGION=eu-central-1

MODULAR_API_SECRET_KEY=your-secure-passphrase
MODULAR_API_MODE=saas
MODULAR_API_CALLS_PER_SECOND_LIMIT=30
MODULAR_API_MIN_CLI_VERSION=2.0
MODULAR_API_ENABLE_PRIVATE_MODE=false

# Logs configuration
MODULAR_API_SERVER_LOG_LEVEL=INFO
MODULAR_API_CLI_LOG_LEVEL=INFO
MODULAR_API_LOG_PATH=/var/log/modular
```

Enable and start service:

```
systemctl daemon-reload
systemctl enable m3-modular-api.service
```

```
systemctl start m3-modular-api.service
systemctl status m3-modular-api.service
```

Verify Server Health

```
# Using curl
curl http://localhost:8086/integration/health

# Using modular CLI
modular describe
```

Access Swagger Documentation

Open browser: `http://localhost:8086/swagger`

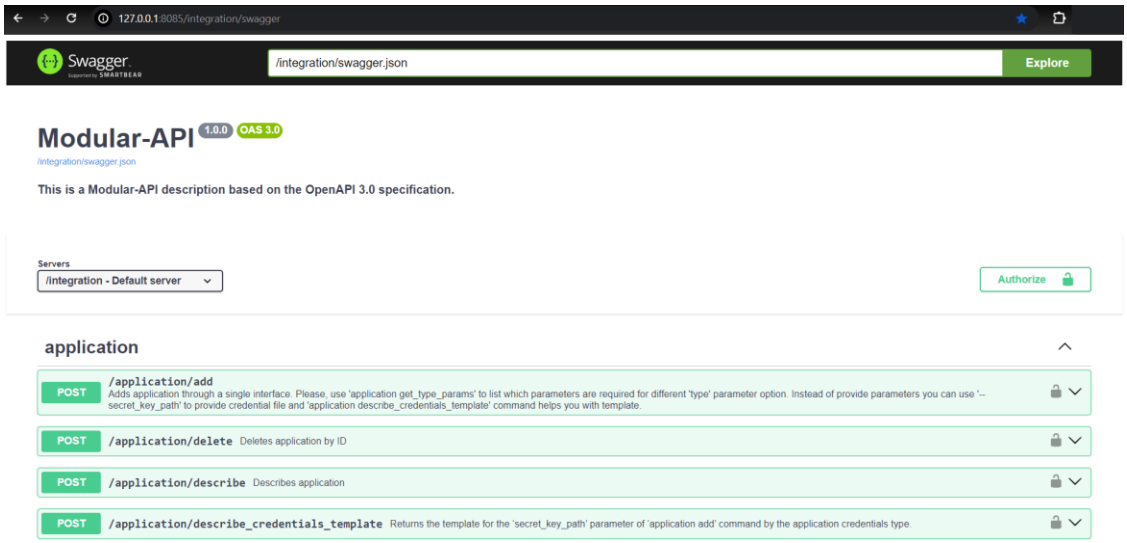

Figure 1 - Swagger

Using API Clients

If you use Modular-CLI, please see the Modular-CLI README.

If you use another API client (for example, Postman):

1. Login and Get API Meta

Path: Modular-API server http
Resource: “login”
Authorization type: “Basic auth”

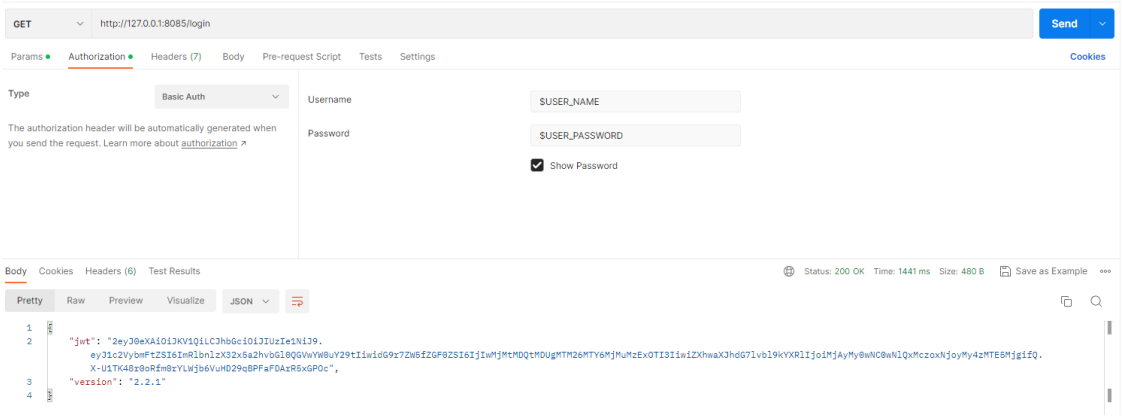

Figure 2 - Authorization

For retrieving commands meta pass parameter `meta`, value is `true`

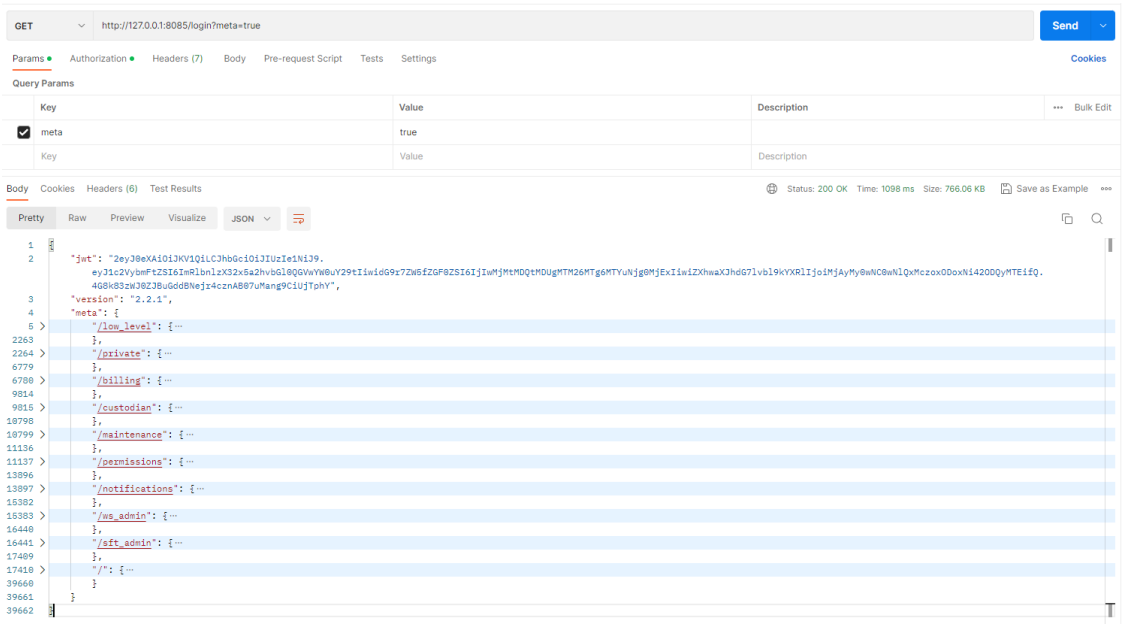

Figure 3 - Authorization

2. Use the API Meta

Use the API meta to find the desired command and create a new request for command execution:

- **Path**: `meta` > `$module_name` > `body` > `$command_name` > `body` > `route` > `path`
- **Method**: `meta` > `$module_name` > `body` > `$command_name` > `body` > `route` > `method`
- **Authorization**: Use `Basic auth` with the username and password, or `Bearer Token` with the token from the `jwt` property in the API meta

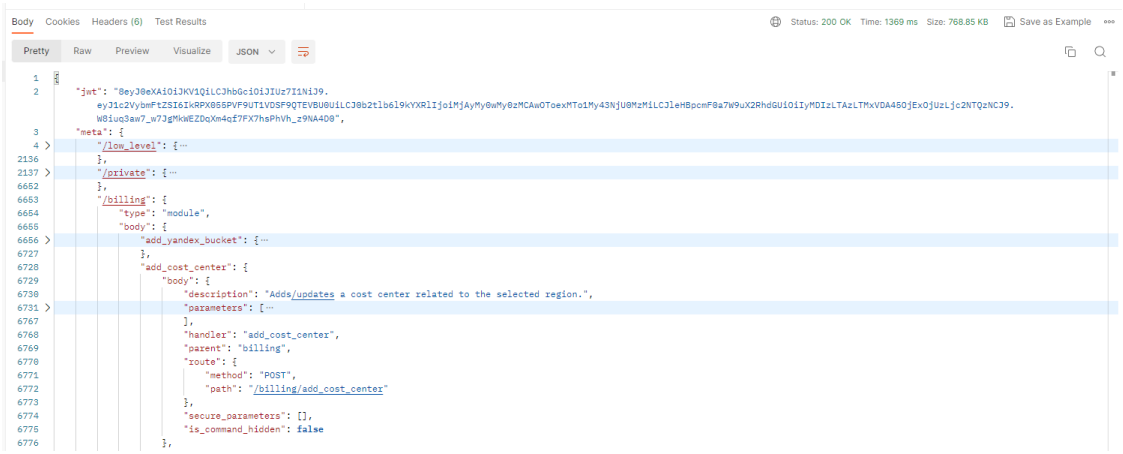

Figure 4 - Meta

Alternatively, Swagger can be used instead of the API meta.

Content ↑

9. Modular API Schema

Architecture Diagram

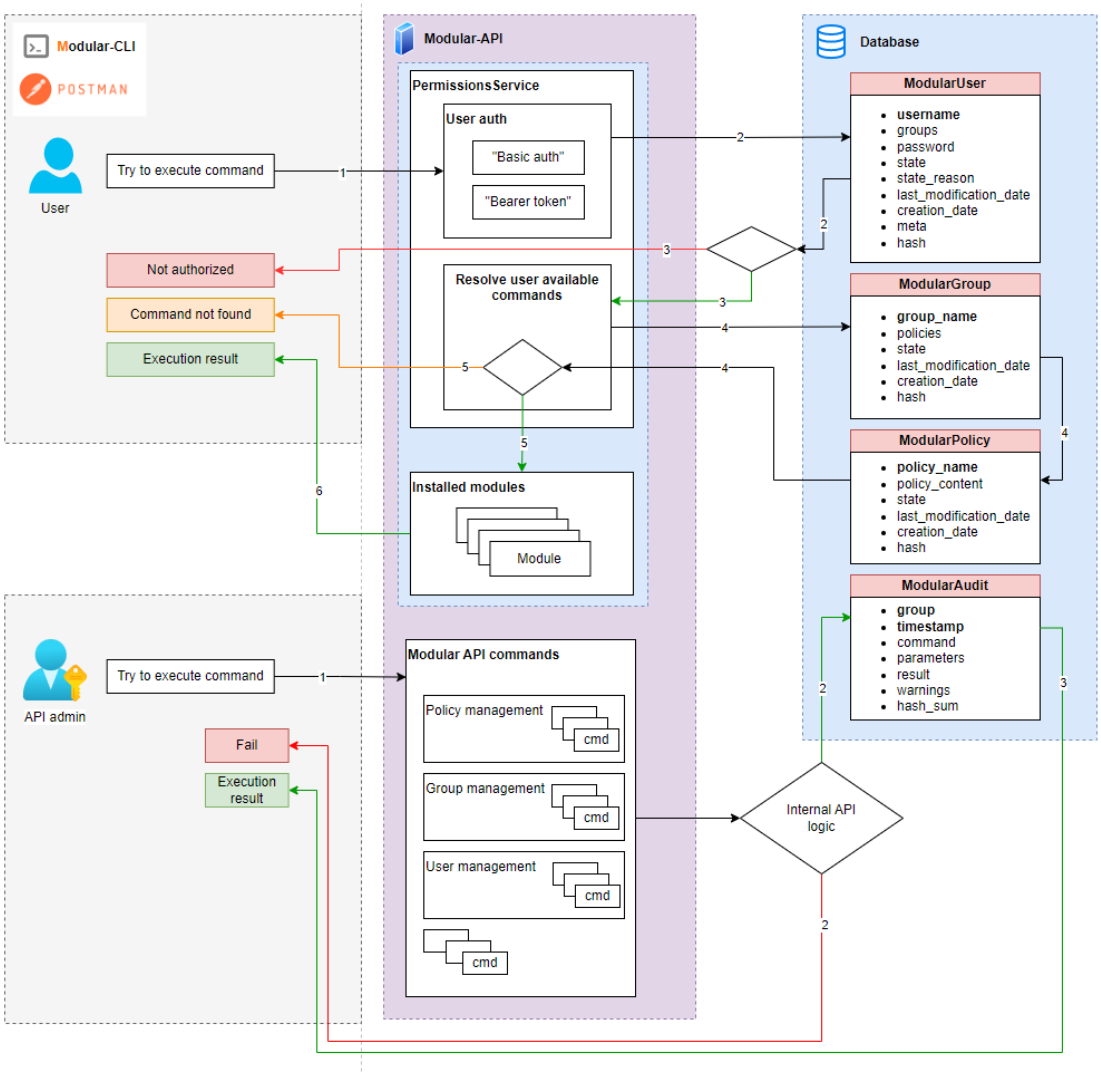

Figure 5 - Schema

Request Flow

For Regular User:

1. User executes command via CLI: `modular <module> <command> --param value`
2. Modular API receives request with JWT token
3. **Authentication**: Validate JWT token
4. **User Lookup**: Retrieve user from database
5. **Policy Resolution**:
    - Get user's groups
    - Collect all policies from groups
    - Merge policies (Deny takes precedence)
6. **Authorization**: Check if command is allowed by policy
7. **Pre-validation**: Validate parameters and meta attributes
8. **Execution**: Forward request to module backend

9. **Audit**: Log successful execution
10. **Response**: Return result to user

For API Administrator:

1. Admin executes management command: `modular policy add ...`
2. **Authentication & Authorization**: Same as regular user
3. **Execution**: Perform management operation
4. **Audit**: Log operation if successful
5. **Response**: Return result

Database Schema

Users Collection

```
{
  "username": "Some_User",
  "password": "06e8e21534119493",
  "groups": ["admin_group"],
  "state": "activated",
  "creation_date": "2025-04-22T10:17:14.224720+00:00",
  "last_modification_date": "2025-08-02T15:56:12.871720+00:00",
  "meta": {
    "allowed_values": {
      "region": ["eu-central-1", "eu-west-1"],
      "tenant_name": ["production-aws-eu"]
    },
    "aux_data": {
      "service_name": {
        "MOBILE": "MOB",
        "DIAL": "AI"
      }
    }
  },
  "hash": "ff591ae23a1874bb"
}
```

Field Descriptions:

Field	Description
username	Unique user identifier
password	Hashed password string
groups	Array of group names the user belongs to
state	User state (e.g., activated, blocked)
creation_date	ISO 8601 timestamp of user creation
last_modification_date	ISO 8601 timestamp of last modification
meta.allowed_values	Parameter restrictions (e.g., allowed regions, tenants)
meta.aux_data	Additional custom data (e.g., service mappings)
hash	Calculated hash for integrity verification

Groups Collection

```
{
  "group_name": "admin_group",
  "policies": ["admin_policy"],
  "state": "activated",
```

```
  "creation_date": "2025-04-17T18:14:03.607970+00:00",
  "hash": "e5f0defefc7b5794"
}
```


Field Descriptions:

Field	Description
group_name	Unique group identifier
policies	Array of policy names attached to this group
state	Group state (e.g., activated)
creation_date	ISO 8601 timestamp of group creation
hash	Calculated hash for integrity verification

Policies Collection


```
{
  "policy_name": "policy-chef",
  "policy_content": "[{\"Description\":\"Chef l3support
policy\",\"Effect\":\"Allow\",\"Module\":\"chef\",\"Resources\":[\"add_configuration\",\"assig
n_configuration\",\"delete_client\",\"delete_configuration\",\"describe_configuration\",\"get_
client\",\"unassign_configuration\",\"update_configuration\",\"update_role\"]}]",
  "state": "activated",
  "creation_date": "2025-01-11T15:55:07.445114+00:00",
  "last_modification_date": "2025-04-17T18:11:28.280355+00:00",
  "hash": "1c9bfd9da1c223ce"
}
```


Field Descriptions:

Field	Description
policy_name	Unique policy identifier
policy_content	JSON string containing policy rules
state	Policy state (e.g., activated)
creation_date	ISO 8601 timestamp of policy creation
last_modification_date	ISO 8601 timestamp of last policy update
hash	Calculated hash for integrity verification

Policy Content Structure (when parsed from JSON string):

```
[
  {
    "Description": "Optional description",
    "Effect": "Allow",
    "Module": "chef",
    "Resources": [
      "add_configuration",
      "assign_configuration"
    ]
  }
]
```

Audit Collection


```
{
  "warnings": [],
```

```
  "group": "user",
  "timestamp": "2025-04-22T10:17:14.229779+00:00",
  "command": "add",
  "parameters": "{\"username\": \"Some_User\", \"group\": [\"admin_group\"], \"password\":
\"*****\"}",
  "result": "User 'Some_User' has been successfully activated.\r\nUser added to the following
group(s):\r\nadmin_group",
  "hash": "85255ee00a11679c"
}
```


Field Descriptions:

Field	Description
warnings	Array of warning messages (if any)
group	Command group (module) that was executed
timestamp	ISO 8601 timestamp of command execution
command	Command name that was executed
parameters	JSON string containing command parameters (passwords masked)
result	Execution result message
hash	Calculated hash for integrity verification

Tokens Collection


```
{
  "u": "Some_User",
  "v": "f2bfa7c051198db523f00597d143447abd449376a5ed5a716d19d80773a86e78"
}
```


Field Descriptions:

Field	Description
u	Username
v	JWT token hash value

Content ↑

10. Annexes

Annex 1: Common Use Cases

This annex provides real-world examples and workflows for common Modular API tasks.

- Use Case 1: Initial System Setup
- Use Case 2: Testing Permissions with Policy Simulator
- Use Case 3: Managing Regional Access
- Use Case 4: Audit and Compliance
- Use Case 5: Backup and Restore
- Use Case 6: Usage Statistics

Use Case 1: Initial System Setup

Scenario: Complete Modular API environment setup from scratch.

Step 1: Install Modular API

```
# Create virtual environment
python3.10 -m venv /usr/local/project/modular/.api_venv
source /usr/local/project/modular/.api_venv/bin/activate

# Install Modular API
pip install -e /usr/local/project/modular/m3-modular-admin/

# Verify installation
modular --version
```

Step 2: Install All M3Admin Modules

```
#!/bin/bash

M3ADMIN_FOLDER="/usr/local/project/modular/m3admin"

echo "Installing m3admin modules..."
modular install --module_path ${M3ADMIN_FOLDER}/src
modular install --module_path ${M3ADMIN_FOLDER}/billing
modular install --module_path ${M3ADMIN_FOLDER}/chef
modular install --module_path ${M3ADMIN_FOLDER}/lowlevel
modular install --module_path ${M3ADMIN_FOLDER}/maintenance
modular install --module_path ${M3ADMIN_FOLDER}/notifications
modular install --module_path ${M3ADMIN_FOLDER}/permissions

# Verify installation
modular describe
```

Step 3: Create Policies

```
#!/bin/bash

POLICY_PATH="/usr/local/project/modular/conf/modular_policies/m3admin_policies"

# Root module policies
modular policy add --policy "m3admin-administrator" \
  --policy_path "${POLICY_PATH}/m3admin_root_module_policies/administrator_policy.json"

modular policy add --policy "m3admin-cicd" \
```

```
  --policy_path "${POLICY_PATH}/m3admin_root_module_policies/cicd_policy.json"

modular policy add --policy "m3admin-l3support" \
  --policy_path "${POLICY_PATH}/m3admin_root_module_policies/l3support_policy.json"

modular policy add --policy "m3admin-m3server" \
  --policy_path "${POLICY_PATH}/m3admin_root_module_policies/m3server_policy.json"

modular policy add --policy "m3admin-readonly" \
  --policy_path "${POLICY_PATH}/m3admin_root_module_policies/readonly_policy.json"

# Billing module policies
modular policy add --policy "billing-administrator" \
  --policy_path "${POLICY_PATH}/m3admin_billing_module_policies/administrator_policy.json"

modular policy add --policy "billing-billing" \
  --policy_path "${POLICY_PATH}/m3admin_billing_module_policies/billing_policy.json"

modular policy add --policy "billing-cicd" \
  --policy_path "${POLICY_PATH}/m3admin_billing_module_policies/cicd_policy.json"

modular policy add --policy "billing-l3support" \
  --policy_path "${POLICY_PATH}/m3admin_billing_module_policies/l3support_policy.json"

modular policy add --policy "billing-m3server" \
  --policy_path "${POLICY_PATH}/m3admin_billing_module_policies/m3server_policy.json"

modular policy add --policy "billing-readonly" \
  --policy_path "${POLICY_PATH}/m3admin_billing_module_policies/readonly_policy.json"

# Chef module policies
modular policy add --policy "chef-administrator" \
  --policy_path "${POLICY_PATH}/m3admin_chef_module_policies/administrator_policy.json"

modular policy add --policy "chef-cicd" \
  --policy_path "${POLICY_PATH}/m3admin_chef_module_policies/cicd_policy.json"

modular policy add --policy "chef-l3support" \
  --policy_path "${POLICY_PATH}/m3admin_chef_module_policies/l3support_policy.json"

modular policy add --policy "chef-readonly" \
  --policy_path "${POLICY_PATH}/m3admin_chef_module_policies/readonly_policy.json"

# Lowlevel module policies
modular policy add --policy "lowlevel-administrator" \
  --policy_path "${POLICY_PATH}/m3admin_low_level_module_policies/administrator_policy.json"

modular policy add --policy "lowlevel-billing" \
  --policy_path "${POLICY_PATH}/m3admin_low_level_module_policies/billing_policy.json"

modular policy add --policy "lowlevel-cicd" \
  --policy_path "${POLICY_PATH}/m3admin_low_level_module_policies/cicd_policy.json"

modular policy add --policy "lowlevel-l3support" \
  --policy_path "${POLICY_PATH}/m3admin_low_level_module_policies/l3support_policy.json"

modular policy add --policy "lowlevel-m3server" \
```

```
    --policy_path "${POLICY_PATH}/m3admin_low_level_module_policies/m3server_policy.json"

  # Maintenance module policies
  modular policy add --policy "maintenance-administrator" \
    --policy_path "${POLICY_PATH}/m3admin_maintenance_module_policies/administrator_policy.json"

  modular policy add --policy "maintenance-billing" \
    --policy_path "${POLICY_PATH}/m3admin_maintenance_module_policies/billing_policy.json"

  modular policy add --policy "maintenance-cicd" \
    --policy_path "${POLICY_PATH}/m3admin_maintenance_module_policies/cicd_policy.json"

  modular policy add --policy "maintenance-l3support" \
    --policy_path "${POLICY_PATH}/m3admin_maintenance_module_policies/l3support_policy.json"

  modular policy add --policy "maintenance-readonly" \
    --policy_path "${POLICY_PATH}/m3admin_maintenance_module_policies/readonly_policy.json"

  # Notifications module policies
  modular policy add --policy "notifications-administrator" \
    --policy_path
  "${POLICY_PATH}/m3admin_notifications_module_policies/administrator_policy.json"

  modular policy add --policy "notifications-cicd" \
    --policy_path "${POLICY_PATH}/m3admin_notifications_module_policies/cicd_policy.json"

  modular policy add --policy "notifications-l3support" \
    --policy_path "${POLICY_PATH}/m3admin_notifications_module_policies/l3support_policy.json"

  modular policy add --policy "notifications-m3server" \
    --policy_path "${POLICY_PATH}/m3admin_notifications_module_policies/m3server_policy.json"

  modular policy add --policy "notifications-readonly" \
    --policy_path "${POLICY_PATH}/m3admin_notifications_module_policies/readonly_policy.json"

  # Permissions module policies
  modular policy add --policy "permissions-administrator" \
    --policy_path "${POLICY_PATH}/m3admin_permissions_module_policies/administrator_policy.json"

  modular policy add --policy "permissions-cicd" \
    --policy_path "${POLICY_PATH}/m3admin_permissions_module_policies/cicd_policy.json"

  modular policy add --policy "permissions-m3server" \
    --policy_path "${POLICY_PATH}/m3admin_permissions_module_policies/m3server_policy.json"

  modular policy add --policy "permissions-l3support" \
    --policy_path "${POLICY_PATH}/m3admin_permissions_module_policies/l3support_policy.json"

  modular policy add --policy "permissions-readonly" \
    --policy_path "${POLICY_PATH}/m3admin_permissions_module_policies/readonly_policy.json"

  echo "All policies created successfully"
```

Step 4: Create Groups

```
  #!/bin/bash

  # M3 Server group - inter-service communication
```

```
modular group add --group "m3server" \
  --policy "lowlevel-m3server" \
  --policy "m3admin-m3server" \
  --policy "billing-m3server" \
  --policy "notifications-m3server" \
  --policy "permissions-m3server"

# Billing team group
modular group add --group "billing" \
  --policy "lowlevel-billing" \
  --policy "billing-billing" \
  --policy "maintenance-billing"

# Read-only group
modular group add --group "readonly" \
  --policy "m3admin-readonly" \
  --policy "billing-readonly" \
  --policy "maintenance-readonly" \
  --policy "permissions-readonly" \
  --policy "chef-readonly" \
  --policy "notifications-readonly"

# L3 Support group
modular group add --group "l3support" \
  --policy "lowlevel-l3support" \
  --policy "m3admin-l3support" \
  --policy "billing-l3support" \
  --policy "maintenance-l3support" \
  --policy "notifications-l3support" \
  --policy "permissions-l3support" \
  --policy "chef-l3support"

# CICD group
modular group add --group "cicd" \
  --policy "lowlevel-cicd" \
  --policy "m3admin-cicd" \
  --policy "billing-cicd" \
  --policy "maintenance-cicd" \
  --policy "notifications-cicd" \
  --policy "permissions-cicd" \
  --policy "chef-cicd"

echo "All groups created successfully"
```

Step 5: Create Initial Users

```
#!/bin/bash

# Create admin user
echo "Creating admin user..."
modular user add --username "admin" --group "administrators"

# Create system users
echo "Creating system users..."
modular user add --username "m3server-svc" --group "m3server"
modular user add --username "cicd-bot" --group "cicd"

echo "Setup complete! Save the generated passwords securely."
```

↑ Back to Use Cases

Use Case 2: Testing Permissions with Policy Simulator

Scenario: Verify user permissions before granting access.

```
# Test if user can execute command
modular policy_simulator --user "support_john" --command "admin aws add_image"

# Test group permissions
modular policy_simulator \
  --group "l3support" \
  --command "admin billing export_aws"

# Test policy directly
modular policy_simulator \
  --policy "m3admin-l3support" \
  --command "admin tenant describe"
```

Expected output:

```
Checked for user: admin
Command: admin aws add_image
Status: ALLOW
```

↑ Back to Use Cases

Use Case 3: Managing Regional Access

Scenario: Restrict support engineers to specific regions for compliance.

```
# Create region-restricted support user
modular user add --username "support_eu" --group "l3support"

# Restrict to EU regions only
modular user set_meta_attribute \
  --username "support_eu" \
  --key "region" \
  --value "eu-central-1" \
  --value "eu-west-1" \
  --value "eu-west-2" \
  --value "eu-north-1"

# Verify restrictions
modular user get_meta --username "support_eu"

# Test: This will work
admin tenant describe --region eu-central-1

# Test: This will be blocked
admin tenant describe --region us-east-1
# Error: Value 'us-east-1' not allowed for parameter 'region'
```

↑ Back to Use Cases

Use Case 4: Audit and Compliance

Scenario: Generate monthly compliance report for security audit.

```
#!/bin/bash

OUTPUT_DIR="/opt/reports/audit/$(date +%Y-%m)"
mkdir -p ${OUTPUT_DIR}

# Generate comprehensive audit report
modular audit \
  --from_date 2025-01-01 \
  --to_date 2025-01-31 \
  --limit 10000 \
  --json > ${OUTPUT_DIR}/full_audit.json

# Generate permission changes report
modular audit \
  --group "permissions" \
  --from_date 2025-01-01 \
  --to_date 2025-01-31 \
  --json > ${OUTPUT_DIR}/permissions_changes.json

# Generate policy changes report
modular audit \
  --group "policy" \
  --from_date 2025-01-01 \
  --to_date 2025-01-31 \
  --json > ${OUTPUT_DIR}/policy_changes.json

# Generate failed operations report
modular audit \
  --invalid \
  --from_date 2025-01-01 \
  --to_date 2025-01-31 \
  --json > ${OUTPUT_DIR}/failed_operations.json

# Generate user activity summary
modular user describe --json > ${OUTPUT_DIR}/users_snapshot.json
modular group describe --json > ${OUTPUT_DIR}/groups_snapshot.json
modular policy describe --json > ${OUTPUT_DIR}/policies_snapshot.json

echo "Compliance reports generated in ${OUTPUT_DIR}"
```

↑ Back to Use Cases

Use Case 5: Backup and Restore

Scenario: Backup all Modular API configuration for disaster recovery.

Backup Script:

```
#!/bin/bash

BACKUP_DIR="/backup/modular/$(date +%Y%m%d_%H%M%S)"
mkdir -p ${BACKUP_DIR}

echo "Starting Modular API backup..."

# Export policies
echo "Backing up policies..."
for policy in $(modular policy describe --json | jq -r '.[].policy_name'); do
```

```
  modular policy describe --policy "$policy" --json > "${BACKUP_DIR}/policy_${policy}.json"
done

# Export groups
echo "Backing up groups..."
modular group describe --json > ${BACKUP_DIR}/groups.json

# Export users (without passwords)
echo "Backing up users..."
modular user describe --json > ${BACKUP_DIR}/users.json

# Export modules info
echo "Backing up modules..."
modular describe --json > ${BACKUP_DIR}/modules.json

# Backup configuration files
echo "Backing up configuration..."
cp -r /usr/local/project/modular/conf ${BACKUP_DIR}/
cp /usr/local/project/modular/m3-modular-admin/modular_api/.env ${BACKUP_DIR}/

# Create tarball
tar -czf "${BACKUP_DIR}.tar.gz" -C $(dirname ${BACKUP_DIR}) $(basename ${BACKUP_DIR})
rm -rf ${BACKUP_DIR}

echo "Backup completed: ${BACKUP_DIR}.tar.gz"
```

Restore Script:

```
#!/bin/bash

if [ -z "$1" ]; then
  echo "Usage: $0 <backup_tarball>"
  exit 1
fi

BACKUP_FILE="$1"
RESTORE_DIR="/tmp/modular_restore_$$"

# Extract backup
mkdir -p ${RESTORE_DIR}
tar -xzf ${BACKUP_FILE} -C ${RESTORE_DIR}

BACKUP_CONTENT=$(find ${RESTORE_DIR} -mindepth 1 -maxdepth 1 -type d)

echo "Restoring Modular API from backup..."

# Restore policies
echo "Restoring policies..."
for policy_file in ${BACKUP_CONTENT}/policy_*.json; do
  policy_name=$(basename "$policy_file" | sed 's/^policy_//' | sed 's/.json$//')
  modular policy add --policy "$policy_name" --policy_path "$policy_file" 2>/dev/null || \
  modular policy update --policy "$policy_name" --policy_path "$policy_file"
done

# Restore groups
echo "Restoring groups..."
for group in $(jq -r '.[].group_name' ${BACKUP_CONTENT}/groups.json); do
  policies=$(jq -r ".[] | select(.group_name==\"$group\") | .policies[]"
```

```
${BACKUP_CONTENT}/groups.json)
  policy_args=""
  for policy in $policies; do
    policy_args="$policy_args --policy $policy"
  done
  modular group add --group "$group" $policy_args
done

echo "Restore completed. Please recreate users manually due to password security."

# Cleanup
rm -rf ${RESTORE_DIR}
```

↑ Back to Use Cases

Use Case 6: Usage Statistics

Scenario: Generate usage statistics for capacity planning.

```
#!/bin/bash

OUTPUT_DIR="/opt/reports/stats"
mkdir -p ${OUTPUT_DIR}

# Get current month stats
modular get_stats \
  --from_month "2025-01" \
  --to_month "2025-02" \
  --path ${OUTPUT_DIR}

# Display stats in terminal
modular get_stats \
  --from_month "2025-01" \
  --to_month "2025-02" \
  --display_table

# Generate quarterly report
modular get_stats \
  --from_month "2025-01" \
  --to_month "2025-04" \
  --path ${OUTPUT_DIR} \
  --json
```

↑ Back to Use Cases

Annex 2: Developers Guide

This annex provides guidance for developers extending Modular API with custom modules.

- Module Development Structure
- api_module.json Template
- pyproject.toml Template
- Command File Example
- Installing Custom Module
- Creating Module Policies
- Module Development Best Practices

Module Development Structure

Required Files:

1. **api_module.json** - Module metadata
2. **setup.py** or **pyproject.toml** - Python package configuration
3. **Command files** - Python files with Click commands

Example Module Structure:

```
my-custom-module/
├── api_module.json
├── pyproject.toml
├── CHANGELOG.md
├── README.md
├── custom_group/
│   ├── __init__.py
│   ├── custom.py                    # Root commands
│   └── resources.py                 # Subgroup: resources deploy commands
└── custom_handler/
    ├── __init__.py
    └── custom_group_handler.py      # Business logic
```

↑ Back to Developers Guide

api_module.json Template

```
{
  "module_name": "my-custom-module",
  "cli_path": "my_module.main",
  "mount_point": "/custom",
  "dependencies": [
    {
      "module_name": "m3admin",
      "min_version": "3.150.0"
    }
  ]
}
```

↑ Back to Developers Guide

pyproject.toml Template

```
[build-system]
requires = ["setuptools"]
build-backend = "setuptools.build_meta"

[project]
name = "custom"
description = "custom module"
requires-python = ">=3.10"
dynamic = ["version"]
dependencies = []

[project.scripts]
custom = "custom_group.custom:custom"

[tool.setuptools.dynamic]
version = {attr = "__version__.__version__"}
```

```
[tool.setuptools.packages.find]
where = ["."]

[tool.pyright]
include = ["custom_group", "custom_handler"]
exclude = ["**/__pycache__"]
pythonVersion = "3.10"
reportIncompatibleMethodOverride = "warning"
```

↑ Back to Developers Guide

Command File Example (custom.py)

```
import click

@click.group()
def cli():
    """
    My Custom Module commands
    """
    pass

@cli.command()
@click.option('--resource-id', required=True, help='Resource ID')
@click.option('--table', is_flag=True, help='Output as table')
@click.option('--json', 'output_json', is_flag=True, help='Output as JSON')
def describe(resource_id, table, output_json):
    """
    Describe a resource
    """
    # Implementation here
    click.echo(f"Describing resource: {resource_id}")

@cli.group()
def resources():
    """
    Resource management commands
    """
    pass

@resources.command()
@click.option('--name', required=True, help='Resource name')
def create(name):
    """
    Create a new resource
    """
    click.echo(f"Creating resource: {name}")

if __name__ == '__main__':
    cli()
```

↑ Back to Developers Guide

Installing Custom Module

```
# Install module
modular install --module_path /path/to/my-custom-module

# Verify installation
```

```
modular describe

# Test commands
modular describe  # Should show your module
```

↑ Back to Developers Guide

Creating Module Policies

custom-module-admin-policy.json:

```
[
  {
    "Effect": "Allow",
    "Module": "my-custom-module",
    "Resources": ["*"]
  }
]
```

Install policy:

```
modular policy add \
  --policy "custom-module-admin" \
  --policy_path "/path/to/custom-module-admin-policy.json"

modular group add_policy \
  --group "administrators" \
  --policy "custom-module-admin"
```

↑ Back to Developers Guide

Module Development Best Practices

Practice	Description
Use Click decorators	For command definition
Include –table and –json	Flags for output formatting
Validate inputs	Before processing
Use meaningful error messages	Clear, actionable error messages
Document all commands	With docstrings
Follow naming conventions	groupname.py for groups, groupname_subgroupname.py
Include dependencies	In api_module.json
Version your module	Properly using semantic versioning
Test thoroughly	Before deployment

↑ Back to Developers Guide

Content ↑

11. Project Information

Project Links

Resource	URL
Source Code	https://github.com/epam/modular-api/tree/main
Documentation	https://github.com/epam/modular-api/blob/main/README.md
Changelog	https://github.com/epam/modular-api/blob/main/CHANGELOG.md

Related Projects

Project	URL
Modular-CLI	https://github.com/epam/modular-cli
Modular-SDK	https://github.com/epam/modular-sdk
Modular-CLI-SDK	https://github.com/epam/modular-cli-sdk

Support

Contact	Details
Email	SupportSyndicateTeam@epam.com
Response Time	7 calendar days (5 business days, excluding weekends)
Python Version	3.14

How to Report an Issue

When reporting issues, provide:

1. **Python version**: Modular API requires Python 3.14
2. **Modular API version**: Run `modular describe`
3. **Clear description**: Concise issue description
4. **Steps to reproduce**: Detailed reproduction steps
5. **Error messages**: Complete error output and logs
6. **Environment details**: OS, deployment mode (saas/onprem), database type

Version Information

```
# Check Modular API version
modular describe

# Check Python version
python --version

# Check installed modules
modular describe --json
```

License

Please refer to the project repository for licensing information.

Content ↑

Last Updated: June 2026
Document Version: 1.0.0
Company: EPAM

Version History

Version	Date	Summary
1.0	July 30, 2026	Initial version created

View Changelog: https://git.epam.com/epmc-eoos/m3-modular-admin/-/blob/develop/CHANGELOG.md

Documentation generated for: **modular, version 4.4.0**